Abstract

This report presents an LSTM-based dynamic system identification procedure for estimating actuator displacement and Lorentz force from two chirp-excitation experiments. The objective is to learn a discrete-time, data-driven model that captures the nonlinear temporal relationship between coil current and the actuator responses. The approach is guided by Wang's LSTM-based dynamic system identification framework, especially the use of discrete-time input-output history and series-parallel identification, and by Ogunmolu et al.'s use of deep dynamic neural networks for sequential system-identification data with regression-based loss and separated evaluation data.

The implemented model uses a series-parallel LSTM structure. At each prediction step, a recent history of coil current, displacement, and Lorentz force is used to estimate the next displacement and force. Because only two chirp experiments are available, both records are divided into train, validation, and test blocks rather than using one full chirp only for training and the other only for testing. The architecture uses two LSTM layers with 64 hidden units, a 120-sample history window, a nonlinear prediction head, weighted multi-output mean-squared-error loss, and regularization tools such as dropout, weight decay, learning-rate scheduling, gradient clipping, and early stopping. The numerical settings are treated as implementation hyperparameters selected for this specific two-chirp actuator dataset.

1 Purpose and Main Idea

The goal of this work is to identify a dynamic model of the actuator directly from measured data. The model receives recent actuator history and predicts the next-step output. The input signal is coil current I, and the outputs are displacement x and Lorentz force F.

Main system-identification task

The actuator is treated as a nonlinear discrete-time dynamic system. The LSTM learns the mapping

$$\{I(k-119:k), x(k-119:k), F(k-119:k)\} \rightarrow [x(k+1), F(k+1)]. \tag{1}$$

Therefore, the model is not fitting isolated data points; it is learning how recent actuator history affects the next displacement and force.

2 Connection to the Reference Papers

The methodology is based on selected ideas from the two reference papers. The objective is not to reproduce every method in those studies, but to apply the relevant system-identification concepts to the actuator dataset.

2.1 Connection to Wang's LSTM Dynamic Identification Paper

Wang presents nonlinear dynamic systems in discrete-time form and discusses LSTM-based system identification using input-output history. The most relevant idea for this simulation is the series-parallel identification structure, where measured output history is used together with input history during training [1].

How is Wang's idea used in this work?

Wang's formulation uses past inputs and past outputs to estimate the dynamic response. In this simulation, the same idea is used with actuator variables:

$$\text{input history: } [I(k), x(k), F(k)], \qquad \text{target: } [x(k+1), F(k+1)]. \tag{2}$$

The similarity is the use of discrete-time input-output history and measured output feedback during training. The difference is that this implementation predicts two outputs and does not implement Wang's convex multi-LSTM cluster.

2.2 Connection to Ogunmolu et al.'s Deep Dynamic Neural Network Paper

Ogunmolu et al. train deep dynamic neural networks on sequential input-output data and evaluate whether the trained model generalizes to separated data [2]. Their work supports the idea that recurrent/deep neural structures can be used for nonlinear system identification when the data are sequential.

How is Ogunmolu et al.'s idea used in this work?

The actuator data are converted into sequential LSTM windows instead of independent static rows. The model is trained by minimizing a regression loss based on prediction error, and performance is evaluated on validation and test windows that are not used for weight updates. This follows the same general training and evaluation approach used in deep dynamic system identification.

3 Dataset and Data Usage

The dataset contains two experimental records named `Chirp_1` and `Chirp_2`. Each record is a time-series experiment of the same actuator under chirp current excitation.

`Chirp_1` and `Chirp_2` are two experimental runs of the same actuator. In each run, a chirp current is applied and the actuator response is recorded over time. They are not two different outputs. Each record contains the input current and the corresponding displacement and Lorentz force response.

3.1 Sampling and LSTM Window Creation

The raw time-series data are resampled to a common time step of

$$\Delta t = 0.002 \text{ s}. \tag{3}$$

This makes one discrete step have the same physical meaning in both chirp experiments.

How many sample times are used for one prediction?

The sequence length is 120 samples. Therefore, one LSTM input example contains 120 consecutive sampled time instants, and the target is the next sample:

$$[I(1:120), x(1:120), F(1:120)] \rightarrow [x(121), F(121)]. \tag{4}$$

Since $\Delta t = 0.002$ s, the input history corresponds to

$$120 \times 0.002 = 0.24 \text{ s}. \tag{5}$$

This 0.24 s history window is long enough to provide recent actuator memory and phase information, while remaining short enough to keep training cost reasonable for only two chirp experiments.

3.2 Train, Validation, and Test Strategy

The two chirp records are kept as separate experimental time series during preprocessing. The data split is then performed inside each chirp using two levels: a block level for organizing the experiment in time, and a window level for creating LSTM training examples.

Two-level data organization used in this simulation

Level 1: block-based splitting. Each chirp is divided into contiguous blocks of 2500 samples. Since the resampled time step is $\Delta t = 0.002$ s, each block represents 5.0 s of actuator response. Each block is split by time into 70% training, 15% validation, and 15% testing.
Level 2: LSTM window creation. After the train/validation/test portions are defined, 120-sample LSTM windows are created inside each portion. Each 120-sample window is used to predict the next sampled displacement and force.

Numerical interpretation of the data split

For one 2500-sample block, the split is approximately

$$2500 = 1750 \text{ (training)} + 375 \text{ (validation)} + 375 \text{ (testing)}. \tag{6}$$

Inside each of these portions, the LSTM examples are formed using a 120-sample history to predict the next sample:

$$1{:}120 \rightarrow 121, \qquad 2{:}121 \rightarrow 122, \qquad 3{:}122 \rightarrow 123, \ldots \tag{7}$$

This procedure keeps the experimental records physically meaningful: windows are created within each chirp and within each split portion, rather than across unrelated parts of the dataset.

4 Model Architecture

The implemented model is a multi-output LSTM identifier. In the default series-parallel mode, the input dimension is three:

$$[I(k), x(k), F(k)], \tag{8}$$

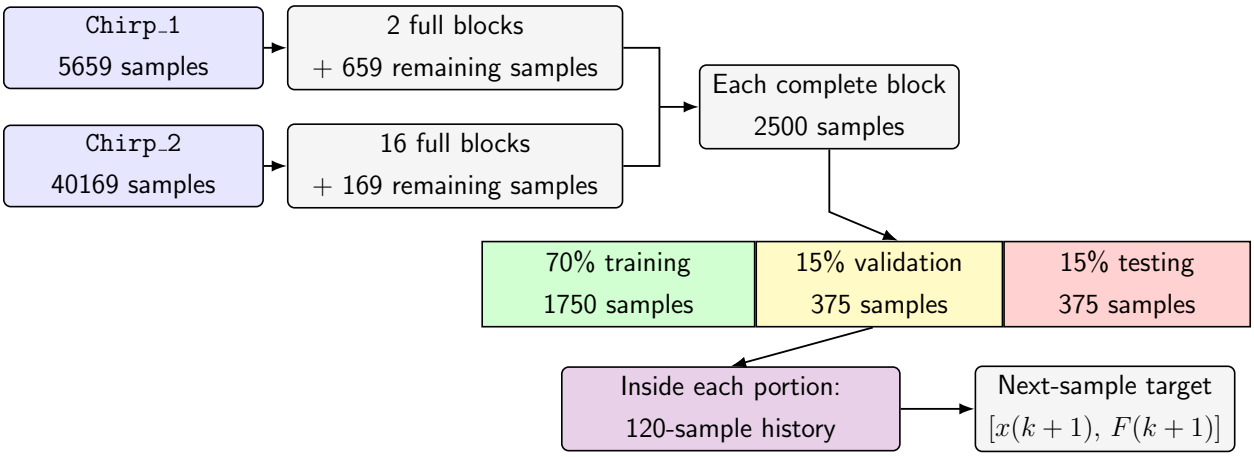

Figure 1: Data usage summary for the LSTM identification procedure.

and the output dimension is two:

$$[\hat{x}(k+1), \hat{F}(k+1)]. \tag{9}$$

Item	Value	reason
Input mode	Series-parallel	Uses current plus measured displacement and force history, following Wang's input-output identification idea.
Sequence length	120 samples	Gives 0.24 s of recent actuator history after resampling.
LSTM depth	2 layers	Provides moderate temporal modeling capacity without making the model too deep for two chirp experiments.
Hidden units	64	Gives enough memory capacity for nonlinear dynamics while limiting overfitting risk.
Prediction head	Linear–tanh–dropout–linear	Adds a small nonlinear mapping from the LSTM memory state to the two physical outputs.
Output	2 variables	Predicts next-step displacement and Lorentz force.

Table 1: High-level architecture summary.

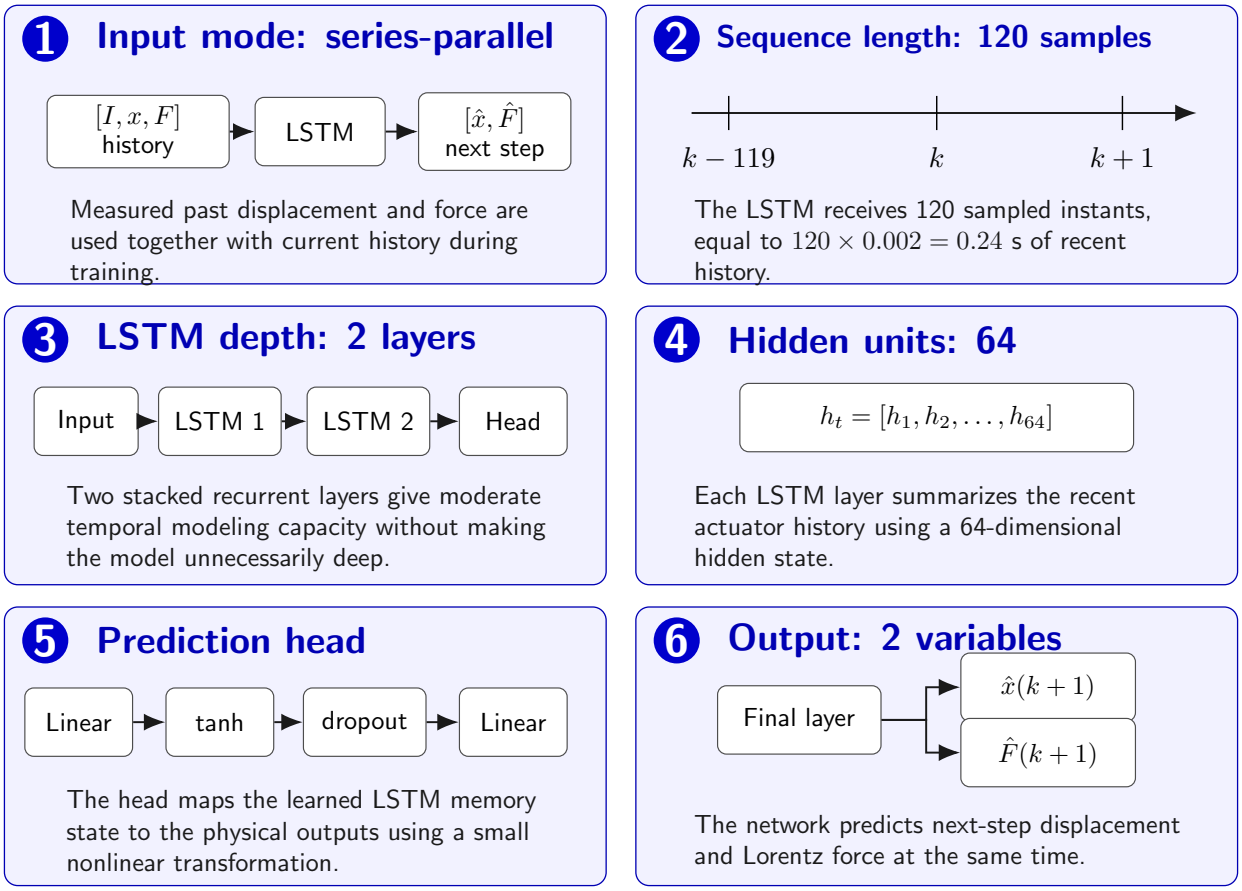

Figure 2: Graphical interpretation of the architecture choices listed in Table 1.

Model parameters vs. engineering hyperparameters

The neural network has two types of quantities:

$$\underbrace{\theta = \{W, b\}}_{\text{learned during training}} \qquad \text{and} \qquad \underbrace{\lambda = \{N_L, N_h, L_{\text{seq}}, \eta, p_{\text{drop}}\}}_{\text{selected before training}}.$$

The network learns the weights and biases θ by minimizing the prediction loss:

$$\theta^* = \arg\min_{\theta} \frac{1}{N} \sum_{j=1}^{N} \left[2.0(\hat{x}_j - x_j)^2 + 1.0(\hat{F}_j - F_j)^2 \right].$$

However, the main model settings λ are selected before training:

$$N_L = 2, \qquad N_h = 64, \qquad L_{\text{seq}} = 120, \qquad \eta = 5 \times 10^{-4}, \qquad p_{\text{drop}} = 0.10.$$

These values were selected as engineering hyperparameters for this dataset. The reference papers motivate the use of LSTM-based dynamic system identification, while the numerical values are chosen based on the size of the available data, the nonlinear actuator behavior, training stability, and overfitting risk.

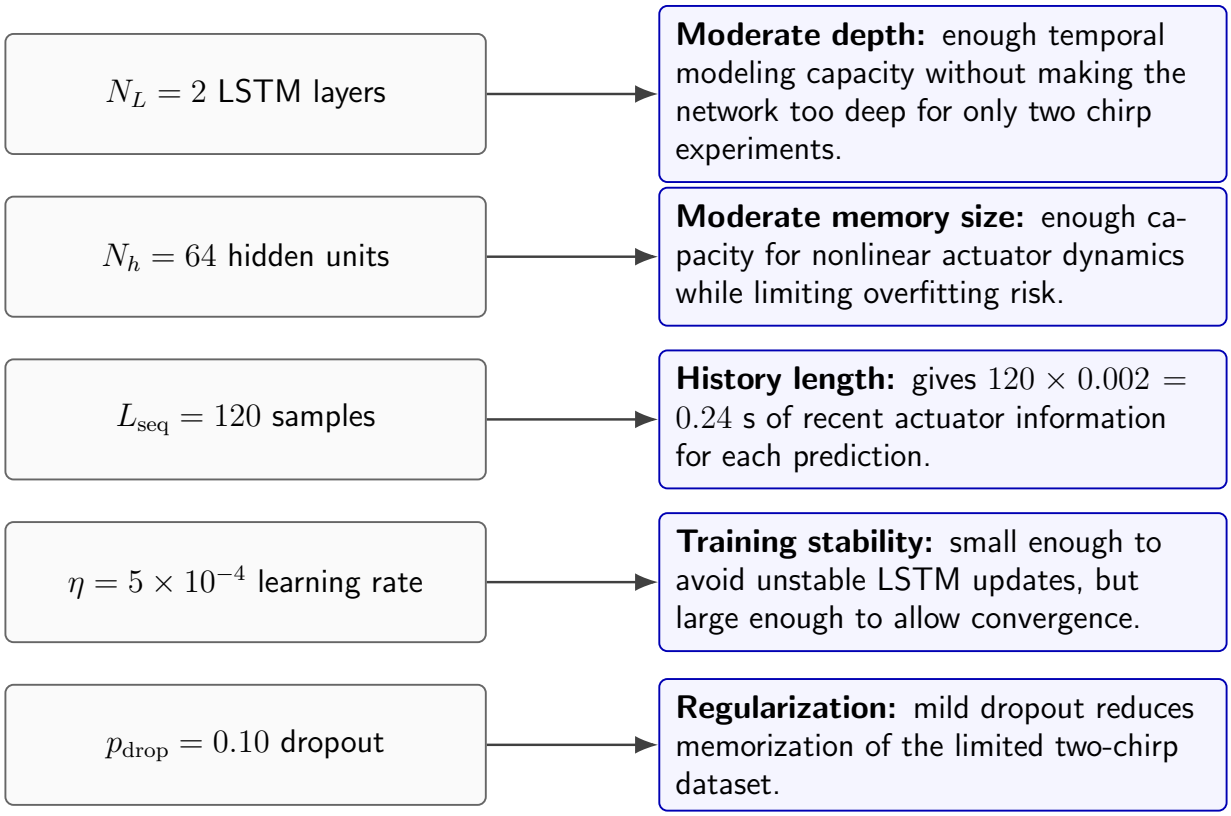

Figure 3: Selected engineering hyperparameters and the high-level reason for each choice. These values were chosen before training based on the available actuator dataset and training behavior.

5 Training Method

The model is trained as a supervised regression problem. Each input window is passed through the LSTM, the prediction head estimates the next displacement and force, and the prediction is compared with the measured target.

5.1 Loss Function

A weighted multi-output MSE loss is used:

$$\mathcal{L} = \frac{1}{N} \sum_{j=1}^{N} \left[2.0(\hat{x}_j - x_j)^2 + 1.0(\hat{F}_j - F_j)^2 \right]. \tag{10}$$

The model predicts two quantities with different physical meanings. Weighted MSE keeps both outputs in the training objective but gives extra attention to displacement, which was the more difficult response to match. This is a practical extension of MSE-based regression to a two-output actuator model.

5.2 Training Settings

The main training settings are summarized in Table 2.

Setting	Value	Reason
Optimizer	AdamW	Stable adaptive optimization with weight decay.
Learning rate	5×10^{-4}	Conservative value for stable LSTM training.
Dropout	0.10	Mild regularization to reduce memorization.
Weight decay	10^{-5}	Penalizes overly large weights.
Gradient clipping	1.0	Prevents large recurrent gradients from destabilizing training.
Early stopping	Validation-based	Keeps the model from continuing after validation performance stops improving.

Table 2: High-level training settings.

AdamW optimizer

AdamW was used to update the trainable LSTM parameters during training. Let

$$\theta = \{W, b\}$$

denote the network weights and biases. At each iteration, AdamW updates the parameters using gradient information from the loss:

$$\theta_{k+1} = \theta_k - \eta\, \widehat{m}_k - \eta\, \lambda_w\, \theta_k.$$

Here, η is the learning rate, $\widehat{m}_k$ represents the adaptive Adam gradient update, and λ_w is the weight-decay coefficient. The adaptive term improves training stability, while the weight-decay term discourages unnecessarily large weights and helps reduce overfitting.

6 Simulation Results

This section presents the main plots generated by the simulation.

6.1 Training and Validation Loss

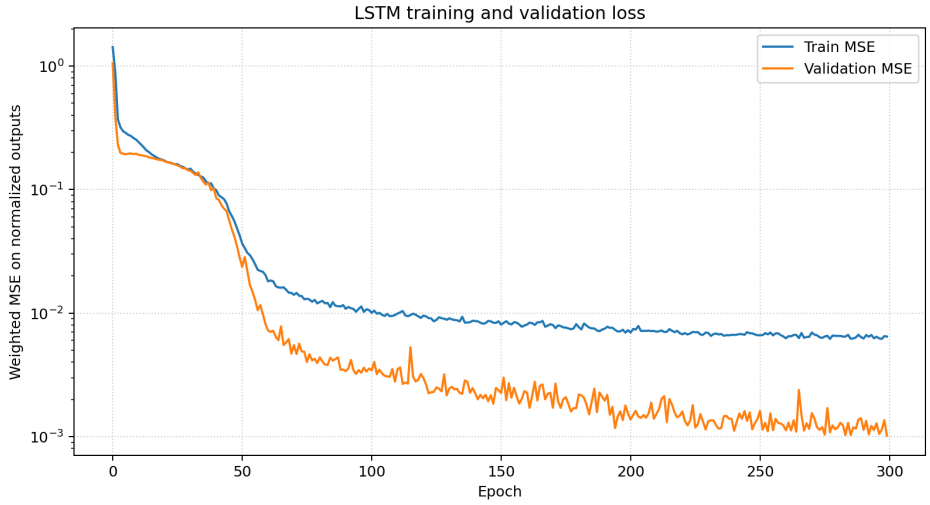

Figure 4: Training and validation loss for the two-chirp LSTM training. The important behavior is that the validation loss decreases and remains stable rather than diverging from the training loss.

6.2 Representative Test Predictions

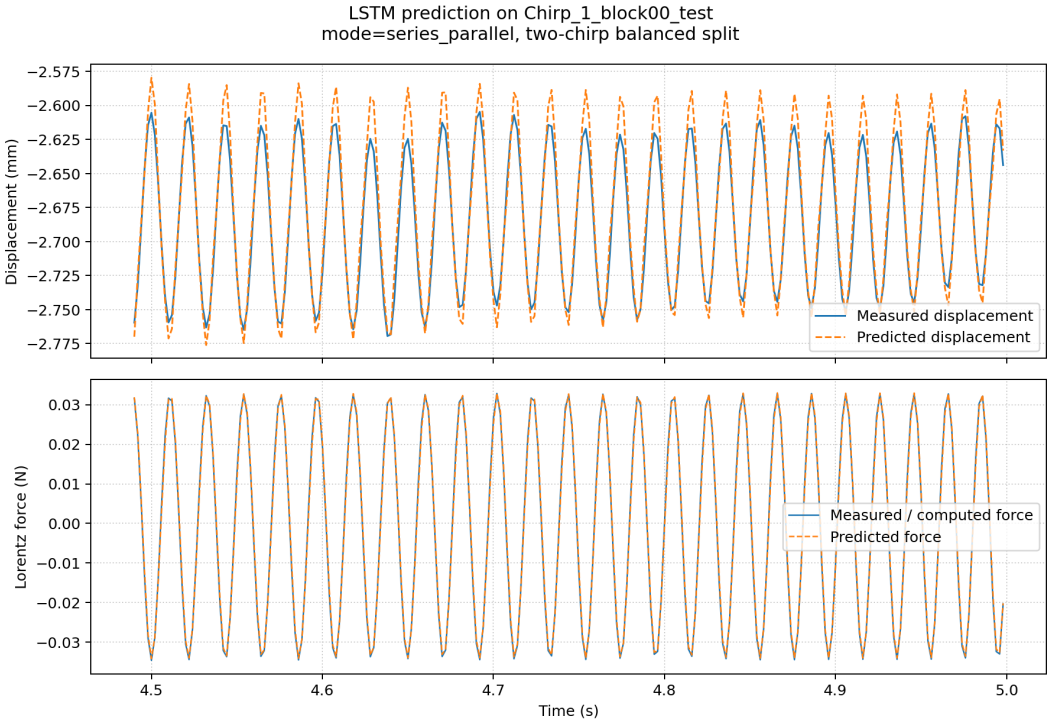

Figure 5: Representative prediction result for `Chirp_1`. The plot compares measured and predicted displacement and Lorentz force on a test block.

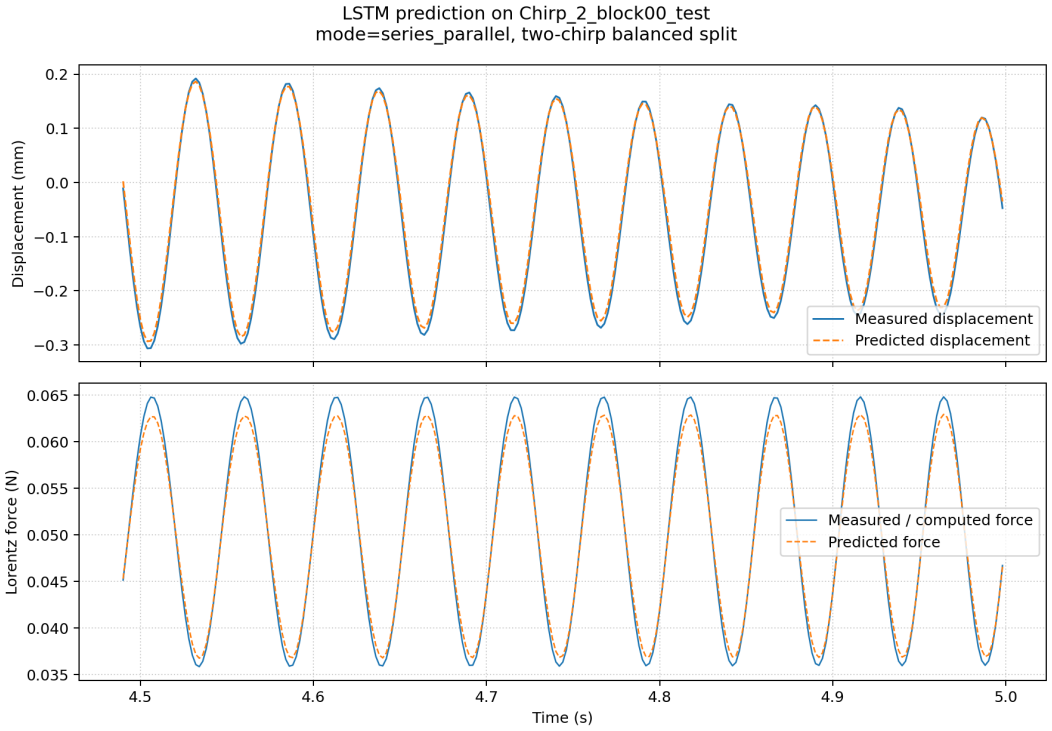

Figure 6: Representative prediction result for `Chirp_2`. The model is evaluated on a test block that was not used for weight updates.

6.3 Representative Prediction Errors

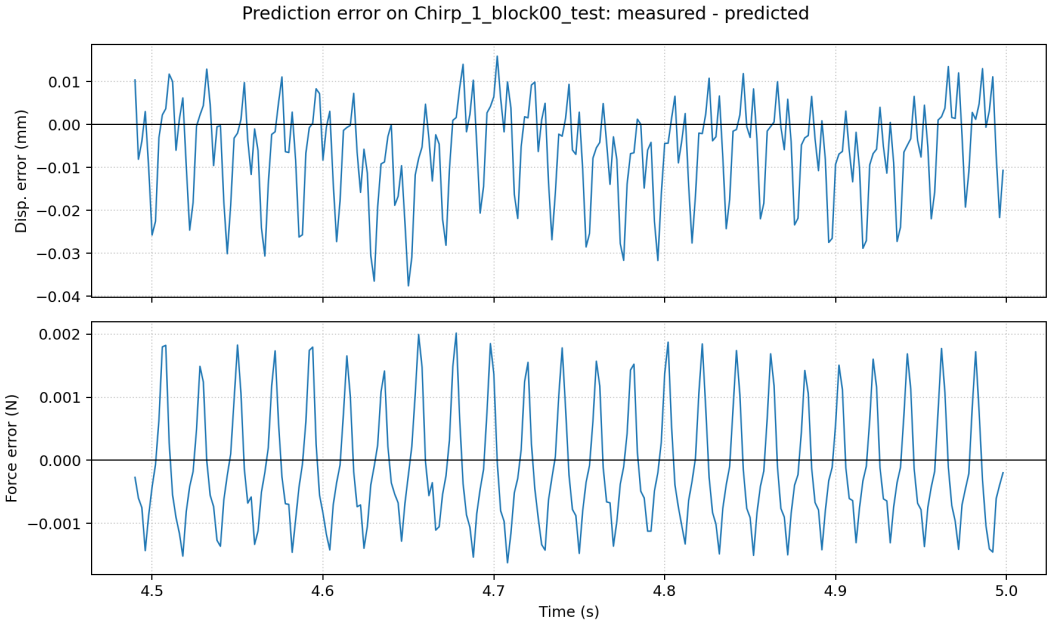

Figure 7: Prediction error for `Chirp_1`, calculated as measured output minus predicted output.

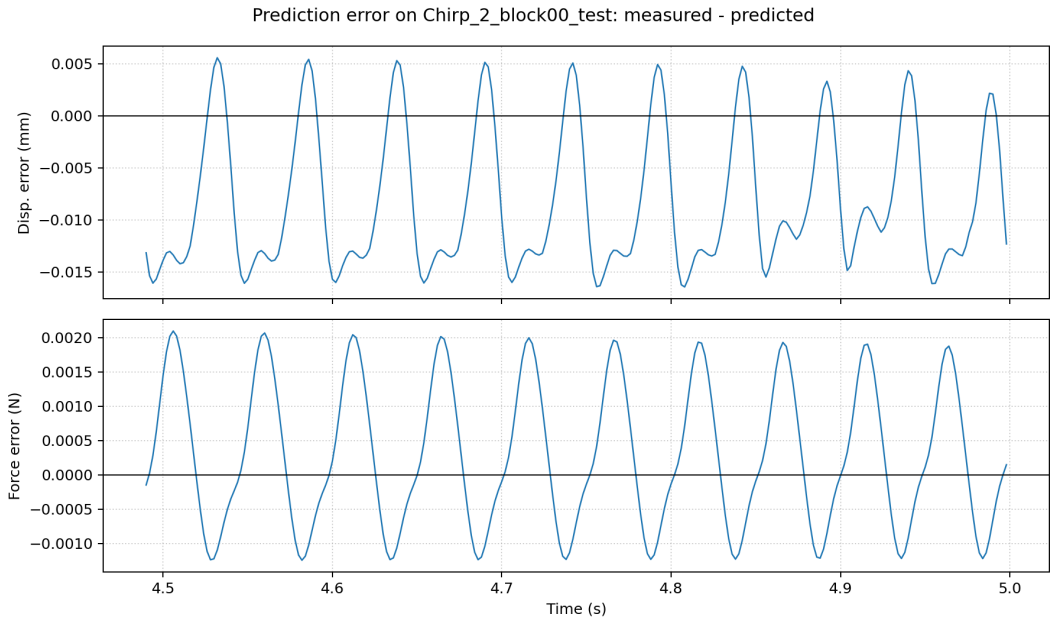

Figure 8: Prediction error for `Chirp_2`, calculated as measured output minus predicted output.

7 Summary of What Was Implemented

Table 3 summarizes the most important connection between paper concepts and the implemented simulation.

Paper concept	Implementation in this simulation	Purpose
Discrete dynamic system [1]	Resampled time-series data indexed by k	Makes the actuator data suitable for next-step prediction.
Series-parallel identification [1]	Input vector includes $[I, x, F]$ history	Uses measured output history during training.
LSTM memory [1]	Two-layer LSTM model	Learns temporal dependencies in the actuator response.
Sequential data learning [2]	Sliding windows of 120 samples	Trains on time histories instead of isolated rows.
MSE regression [2]	Weighted multi-output MSE	Minimizes displacement and force prediction error.
Separated evaluation [2]	Train/validation/test blocks	Evaluates generalization on unseen windows.

Table 3: High-level traceability between reference papers and the implemented model.

8 Conclusion

An LSTM-based series-parallel dynamic identifier was implemented for actuator displacement and Lorentz-force prediction using two chirp experiments. The model uses recent histories of coil current, displacement, and force to predict the next actuator response. The main theoretical direction comes from LSTM-based dynamic system identification and sequential neural-network system iden-

tification. The main hyperparameters were selected to balance learning capacity, training stability, and overfitting risk for the available data. The results show how the trained model follows the measured actuator response on unseen test windows, while the error plots identify the remaining mismatch.

This work uses only two chirp experiments. Therefore, the model should be viewed as an initial data-driven identifier rather than a fully validated actuator model. A stronger future study should include more experiments with different amplitudes, frequencies, operating conditions, and possibly PRBS or step inputs for better generalization testing.

References

[1] Y. Wang, "A New Concept using LSTM Neural Networks for Dynamic System Identification," *Proceedings of the American Control Conference*, Seattle, WA, USA, 2017, pp. 5324–5329.

[2] O. Ogunmolu, X. Gu, S. Jiang, and N. Gans, "Nonlinear Systems Identification Using Deep Dynamic Neural Networks," arXiv:1610.01439, 2016.